smalllabs

Fullstack Web Development

Session 9 | Week 11/Day 1 | 120 min

Laravel - Part05

Instructor: Ms. Liev Nova 090667393

Laravel

Outline

- Database connection
- Migrations
- Creating tables
- Updating tables
- Foreign keys
- Database seeding
- Eloquent ORM
- Laravel models And Naming
- Mass assignment
- Eloquent relationships
- Querying data
- Insert, update, and delete records
- CRUD Operation

Database connection

- **Database in Laravel**
  - A database is used to store application data permanently.
    - Examples:
      - Users
      - Students
      - Companies
      - Tasks
      - Products
      - Orders
  - In Laravel, we usually work with the database through:
    - .env configuration
    - Migration files
    - Models
      - Seeders
      - Eloquent ORM
  - Laravel helps us manage database structure using code, not only by manually creating tables in phpMyAdmin.

Database connection

- Laravel stores database connection settings in the .env file.
  - Example:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=contact_app_db
DB_USERNAME=root
DB_PASSWORD=your_password
```

- `DB_CONNECTION` is the database type.
- `DB_HOST` is the database server address.
- `DB_PORT` is the database port.
- `DB_DATABASE` is the database name.
- `DB_USERNAME` is the database username.
- `DB_PASSWORD` is the database password.

  - After updating `.env`, make sure the database already exists in MySQL.

Database connection

- **Test Database Connection**
  - After creating the database and updating `.env`, run:

```
php artisan migrate
```

  - If the connection is correct, Laravel will create the default tables.
  - Common connection problems:
    - Database name is wrong.
    - MySQL server is not running.
    - Username or password is incorrect.
    - `.env` was changed but the config cache was not cleared.

      - Useful command:

```
php artisan config:clear
```

Migrations

- **What is a Migration?**
  - A migration is a PHP file that describes a database table structure.
  - Migration files are stored in: database/migrations
  - Migrations help developers:
    - Create tables using code.
    - Share database structure with a team.
    - Track database changes.
    - Roll back changes when needed.
  - A migration is like version control for the database structure.

Migrations

- **Migration Structure**
  - A migration usually has two main methods: up and down

```php
public function up()
{
    // Apply database changes
}

public function down()
{
    // Reverse database changes
}
```

- The `up()` method is used to:
  - Create tables
  - Add columns
  - Rename columns
  - Add indexes or foreign keys

- The `down()` method is used to:
  - Drop tables
  - Remove columns
  - Undo the changes from `up()`

Migrations

- **Common Migration Commands**
  - Run all pending migrations:
    ```
    php artisan migrate
    ```
  - Check migration status:
    ```
    php artisan migrate:status
    ```
  - Preview SQL without running it:
    ```
    php artisan migrate --pretend
    ```
  - Rollback the latest migration batch:
    ```
    php artisan migrate:rollback
    ```
  - Drop all tables and run migrations again:
    ```
    php artisan migrate:fresh
    ```
    - Use `migrate:fresh` carefully because it deletes all tables and data.

Migrations

- **Create a Table with Migration**
  - To create a new table, use:

```
php artisan make:migration create_companies_table
```

    - Laravel creates a migration file in: database/migrations
  - The migration name should clearly describe the action: create_companies_table
    - This means: "Create a new table named companies."

Migrations

- **Companies Table Migration**
  - Example:

```php
public function up()
{
    Schema::create('companies', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('address')->nullable();
        $table->string('website')->nullable();
        $table->string('email')->comment('Email of the company');
        $table->timestamps();
    });
}
```

    - Column explanation:
      - `id()` creates an auto-increment primary key.
      - `string()` creates a text column with limited length.
      - `nullable()` allows the column to be empty.
      - `comment()` adds a database note for the column.
      - `timestamps()` creates `created_at` and `updated_at`.

Migrations

- **Run the Migration**
  - After creating the migration, run:

```
php artisan migrate
```

  - Laravel will read the migration file and create the table in the database.
  - If the migration is successful, check the database and you should see: companies
  - The migration file defines the table. The `migrate` command creates the table.

Creating tables

- **Create Tasks Table**
  - Create a migration:

```
php artisan make:migration create_tasks_table
```

```php
public function up()
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('description')->nullable();
        $table->date('due_date')->nullable();
        $table->boolean('status')->default(false);
        $table->timestamps();
    });
}
```

  - Then run:

```
php artisan migrate
```

Creating tables

- **Choosing Column Types**
  - Laravel provides many column types for migrations.
    - Common examples:

```
$table->id();
$table->string('name');
$table->text('description');
$table->integer('quantity');
$table->decimal('price', 8, 2);
$table->boolean('status');
$table->date('due_date');
$table->timestamp('published_at')->nullable();
$table->timestamps();
```

Creating tables

- **Choosing Column Types**

    - Laravel provides many column types for migrations.
        - Choose the column type based on the data you want to store.
            - Examples:
                - Name: `string`
                - Description: `text`
                - Price: `decimal`
                - Status: `boolean`
                - Date: `date`

Updating tables

- **Updating a Table with Migration**

    - Sometimes we need to change an existing table.
        - Examples:
            - Add a new column
            - Remove a column
            - Rename a column
            - Change a column type
            - Add a foreign key

Updating tables

- **Updating a Table with Migration**
    - Sometimes we need to change an existing table.
        - Migration naming examples:
            - add_priority_to_tasks_table
            - remove_priority_from_tasks_table
            - rename_due_date_in_tasks_table
            - add_user_id_to_tasks_table
    - Good migration names make the project easier to understand.

Updating tables

- **Add Column to Existing Table**
  - Create a migration:

```
php artisan make:migration add_priority_to_tasks_table
```

    - Example:

```php
public function up()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->tinyInteger('priority')->default(1)->after('status');
    });
}

public function down()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->dropColumn('priority');
    });
}
```

    - Then run:

```
php artisan migrate
```

Updating tables

- **Rename a Column**
  - Example: rename `due_at` to `due_date`.
    - Create a migration:

```
php artisan make:migration rename_due_at_in_tasks_table
```

    - Example:

```php
public function up()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->renameColumn('due_at', 'due_date');
    });
}

public function down()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->renameColumn('due_date', 'due_at');
    });
}
```

    - The `up()` method applies the change. The `down()` method reverses it.

Foreign Key

- **A foreign key** connects one table to another table.
    - Example: A task belongs to one user.
        - So the `tasks` table needs a `user_id` column.
        - Relationship: users.id -> tasks.user_id
    - Why use foreign keys?
        - Keep related data connected.
        - Prevent invalid data.
        - Make database relationships clear.

Foreign Key

- **Add Foreign Key to Tasks Table**
  - Create a migration:

```
php artisan make:migration add_user_id_to_tasks_table
```

  - Recommended Laravel syntax:
    - `foreignId('user_id')` creates the `user_id` column.
    - - `constrained()` connects it to the `id` column on the `users` table.
    - - `cascadeOnDelete()` deletes related tasks when the user is deleted.

```php
public function up()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->foreignId('user_id')
              ->after('id')
              ->constrained()
              ->cascadeOnDelete();
    });
}

public function down()
{
    Schema::table('tasks', function (Blueprint $table) {
        $table->dropForeign(['user_id']);
        $table->dropColumn('user_id');
    });
}
```

Database seeding

- **What is Database Seeding?**
  - Database seeding means inserting sample or default data into the database.
  - Seeders are useful for:
    - Testing the application
    - Preparing sample data for class
    - Creating default records
    - Quickly resetting project data
  - Seeder files are stored in: database/seeders
  - Create a seeder:

```
php artisan make:seeder CompanySeeder
```

Database seeding

- **Company Seeder Example**
  - Example:

```php
use Illuminate\Support\Facades\DB;

public function run()
{
    $companies = [];

    foreach (range(1, 10) as $index) {
        $companies[] = [
            'name' => "Company $index",
            'address' => "Address $index",
            'website' => "https://company$index.test",
            'email' => "company$index@example.com",
            'created_at' => now(),
            'updated_at' => now(),
        ];
    }

    DB::table('companies')->delete();
    DB::table('companies')->insert($companies);
}
```

- *The seeder creates sample records automatically.*

Database seeding

- **Run Seeders**
  - Run all seeders:

```
php artisan db:seed
```

  - Run one specific seeder:

```
php artisan db:seed --class=CompanySeeder
```

  - Fresh database with seed data:

```
php artisan migrate:fresh --seed
```

    - *Use this when you want to reset the database and insert sample data again.*

Eloquent ORM

- **Run Seeders**
  - Run all seeders:

```
php artisan db:seed
```

  - Run one specific seeder:

```
php artisan db:seed --class=CompanySeeder
```

  - Fresh database with seed data:

```
php artisan migrate:fresh --seed
```

    - *Use this when you want to reset the database and insert sample data again.*

Eloquent ORM

- **What is Eloquent ORM?**
  - Eloquent is Laravel's built-in ORM.
  - ORM means Object-Relational Mapping.
  - It allows us to work with database tables using PHP classes and objects.
    - Example:

```
$companies = Company::all()
```

  - Instead of writing raw SQL:

```
SELECT * FROM companies;
```

    - Eloquent makes database work easier, cleaner, and more readable.

Eloquent ORM

- **Why Use Eloquent?**
  - Eloquent helps developers:
    - Read records from the database.
    - Insert new records.
    - Update existing records.
    - Delete records.
    - Define relationships between tables.
    - Write readable database queries.
  - Example:

```php
$company = Company::find(1);
```

    - *This means: "Find the company with id 1."*

Laravel models And Naming

- Eloquent connects models to database tables using naming conventions.
  - Model name: Company
  - Table name: companies
  - Model rules:
    - Singular form
    - PascalCase
    - Stored in `app/Models`
  - Examples:
    - User -> users
    - Student -> students
  - Table rules:
    - Plural form
    - snake_case
    - Stored in the database
    - Company -> companies
    - Task -> tasks

Laravel models And Naming

- **Create a Model**
  - Create a model:

```
php artisan make:model Company
```

  - Laravel creates: app/Models/Company.php
  - Example:

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Company extends Model
{
    use HasFactory;
}
```

  - The `Company` model represents the `companies` table.

Laravel models And Naming

- **Create Model with Migration and Seeder**
  - Laravel can create related files with one command.

```
php artisan make:model Contact -ms
```

  - This creates:
    - app/Models/Contact.php
    - database/migrations/xxxx_xx_xx_create_contacts_table.php
    - database/seeders/ContactSeeder.php
  - Options:
    - `-m` creates a migration.
    - `-s` creates a seeder.
      - *This is useful when starting a new database feature.*

Laravel models And Naming

- **Example Contact Migration**
  - Example:

```
public function up()
{
    Schema::create('contacts', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->string('phone')->nullable();
        $table->timestamps();
    });
}
```

    - Then run:

```
php artisan migrate
```

      - Now the database has a `contacts` table.

Mass Assignment

- Mass assignment means assigning many model attributes at the same time.
  - Example:

```
Company::create([
    'name' => 'My Company',
    'email' => 'company@example.com',
    'address' => 'Phnom Penh',
]);
```

    - This is useful because it makes insert code shorter.
    - But it can be dangerous if users send fields that should not be saved.
      - Example risk: A user tries to send `is_admin = true`.

Mass Assignment

- **The Fillable Property**
  - The `$fillable` property tells Laravel which fields can be mass assigned.
  - Example:

```php
class Company extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'email',
        'address',
        'website',
    ];
}
```

- Only these fields can be inserted or updated using mass assignment.

- *Use `$fillable` to protect the model from unexpected input.*

Mass Assignment

- **The Guarded Property**
  - The `$guarded` property tells Laravel which fields cannot be mass assigned.
  - Example:

```php
class Company extends Model
{
    use HasFactory;

    protected $guarded = ['id'];
}
```

- This means every field can be mass assigned except `id`.

Mass Assignment

- **The Guarded Property**

    - Common choice:
        - Use `$fillable` when you want strict control.
        - Use `$guarded` when you understand the risk and want shorter model code

Eloquent relationships

- **Eloquent relationships**
  - Database tables are often connected.
    - Examples:
      - A user has one profile.
      - A company has many contacts.
      - A student belongs to one gender.
      - A user can have many roles.
  - Eloquent relationships help us write readable code.

  - Example: $company->contacts;
    - This means: "Get all contacts that belong to this company."

Eloquent relationships

- **One-to-One Relationship**
  - One record is related to one other record.
  - Example: A user has one phone.

```php
class User extends Model
{
    public function phone()
    {
        return $this->hasOne(Phone::class);
    }
}
```

  - Usage: $user->phone;

Eloquent relationships

- **One-to-Many Relationship**
  - One record is related to many other records.
  - Example: A company has many contacts.

```php
class Company extends Model
{
    public function contacts()
    {
        return $this->hasMany(Contact::class);
    }
}
```

  - Usage: $company->contacts;

Eloquent relationships

- **One-to-Many Relationship**
  - One record is related to many other records.
  - Example: A company has many contacts.
    - Opposite relationship:

```php
class Contact extends Model
{
    public function company()
    {
        return $this->belongsTo(Company::class);
    }
}
```

Eloquent relationships

- **Many-to-Many Relationship**
  - Many records can connect to many other records.
  - Example: A user can have many roles. A role can belong to many users.
    - Tables:
      - users
      - roles
      - role_user
  - Model example:

```php
class User extends Model
{
    public function roles()
    {
        return $this->belongsToMany(Role::class);
    }
}
```

  - Usage: $user->roles;

Eloquent relationships

- **Has Many Through Relationship**
  - A model can access another model through an intermediate model.
  - Example: A country has many posts through users.
    - Tables:
      - countries
      - users
      - posts
        - Relationship: Country -> Users -> Posts

```php
class Country extends Model
{
    public function posts()
    {
        return $this->hasManyThrough(Post::class, User::class);
    }
}
```

- Usage: $country->posts;

Eloquent relationships

- **Polymorphic Relationship**
  - One model can belong to more than one type of model.
  - Example: A photo can belong to a post or a product.
  - Tables:
    - photos
    - posts
    - products
      - The `photos` table has:
        - imageable_id
        - imageable_type

```php
class Photo extends Model
{
    public function imageable()
    {
        return $this->morphTo();
    }
}
```

- Usage: $photo->imageable;

Querying data

- **Querying with Tinker**
  - **Laravel Tinker** allows us to test Eloquent commands in the terminal.
    - Start Tinker:

```
php artisan tinker
```

    - Import the model:

```
use App\Models\Company;
```

    - Now we can query the `companies` table using the `Company` model.

Querying data

- **Basic Eloquent Queries**
  - Get all records:
```
Company::all();
```
  - Get the first record:
```
Company::first();
```
  - Find by id:
```
Company::find(2);
```
  - Count records:
```
Company::count();
```
  - Get only selected columns:
```
Company::select('id', 'name', 'email')->get();
```

Querying data

- **Limit, Skip, and Order**
  - Get only 3 records:

```
Company::take(3)->get();
```

  - Skip 2 records and take 3:

```
Company::skip(2)->take(3)->get();
```

  - Order by latest id:

```
Company::orderBy('id', 'desc')->get();
```

  - Get one column:

```
Company::pluck('name');
```

Querying data

- **Where Clauses**
  - Find companies by name:

```php
Company::where('name', 'Company 1')->first();
```

  - Use multiple conditions:

```php
Company::where('id', 9)
    ->where('name', 'Company 9')
    ->get();
```

  - Use comparison operators:

```php
Company::where('id', '>', 3)->get();
```

Querying data

- **Where Clauses**
  - Use `like`:

```
Company::where('name', 'like', 'Company%')->get();
```

  - Check for null value:

```
Company::whereNull('address')->get();
```

Querying data

- **View SQL Query**
  - Sometimes we want to see the SQL query before running it.
  - Example:

```
Company::where('id', '>', 3)->toSql();
```

```
Company::where('name', 'like', 'Company%')->toSql();
```

Insert, update, and delete records

- **Insert a Record**
  - Create a new company:

```
$company = new Company();
$company->name = 'My Company';
$company->email = 'company@example.com';
$company->address = 'Phnom Penh';
$company->website = 'https://example.com';
$company->save();
```

- `new Company()` creates a new model object.
- - We assign values to the object.
- - `save()` inserts the record into the database.

Insert, update, and delete records

- **Insert with Mass Assignment**
  - First, make sure the model has `$fillable`.

```php
protected $fillable = [
    'name',
    'email',
    'address',
    'website',
];
```

  - Then insert:

```php
Company::create([
    'name' => 'New Company',
    'email' => 'new@example.com',
    'address' => 'Siem Reap',
    'website' => 'https://new-company.test',
]);
```

- This method is shorter and commonly used in controllers.

Insert, update, and delete records

- **Update a Record**
  - Method 1: find and save.

```php
$company = Company::find(1);
$company->name = 'Updated Company';
$company->email = 'updated@example.com';
$company->save();
```

  - Method 2: update directly.

```php
Company::where('id', 1)->update([
    'name' => 'Updated Company',
    'email' => 'updated@example.com',
]);
```

    - *Use update when you already know which record should change.*

Insert, update, and delete records

- **Delete a Record**
  - Method 1: find and delete.

```
$company = Company::find(1);
$company->delete();
```

  - Method 2: delete by query.

```
Company::where('id', 2)->delete();
```

  - Method 3: destroy by id.

```
Company::destroy(3);
```

    - *Delete removes records from the database, so use it carefully.*

THANK YOU

For your attention !